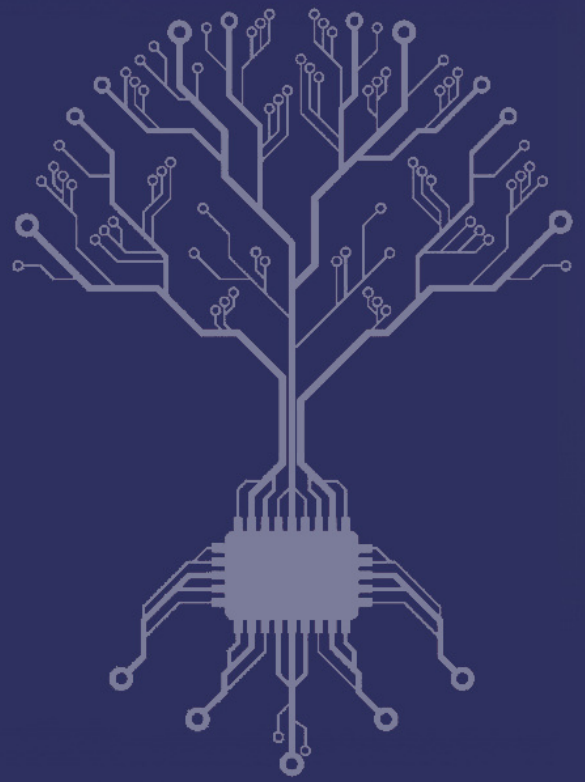
المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵜⵣⵔⴼⵜ ⴰⵏⴰⵎⵓⵔ
ⴷ ⵜⵓⵓⵏⵎⴷⴰ ⴰⵎⵙⵡⴰⵔⴰ ⴷ ⵜⵙⵏⵙⵏⵜ

Base de Données (1/2)

Introduction



Pourquoi une base de données ?

Tout **organisme** manipule chaque jour des données décrivant ses **entités** et leurs **associations**. Ces données doivent être :

- stockées **durablement** ;
- partagées **simultanément** ;
- accessibles **rapidement**,

en garantissant leur **intégrité** et leur **cohérence**.

Exemple : le ministère et les classes préparatoires

Pour les classes préparatoires, le **ministère de l'Éducation nationale** gère :

- des entités **étudiant**, décrites par un code, un nom, un prénom, une moyenne, une ville...
- des entités **centre**, décrites par un code, un nom et une ville ;
- une association **s'inscrire à** : chaque étudiant s'inscrit à un centre, et un centre accueille plusieurs étudiants.



Plutôt que de multiplier les fichiers et les listes papier, le ministère regroupe toutes ces informations dans une **base de données** unique.

Définition

Une **base de données** est un ensemble structuré de données décrivant les **entités** d'un organisme et leurs **associations**.

Définition

Un **SGBD** (Système de Gestion de Bases de Données) est un logiciel (e.g. MySQL, Oracle, SQLite, ...) qui permet de gérer une BD pour définir et manipuler les données, tout en garantissant leur cohérence et intégrité.

Le SGBD utilisé dans la suite est **SQLite**.

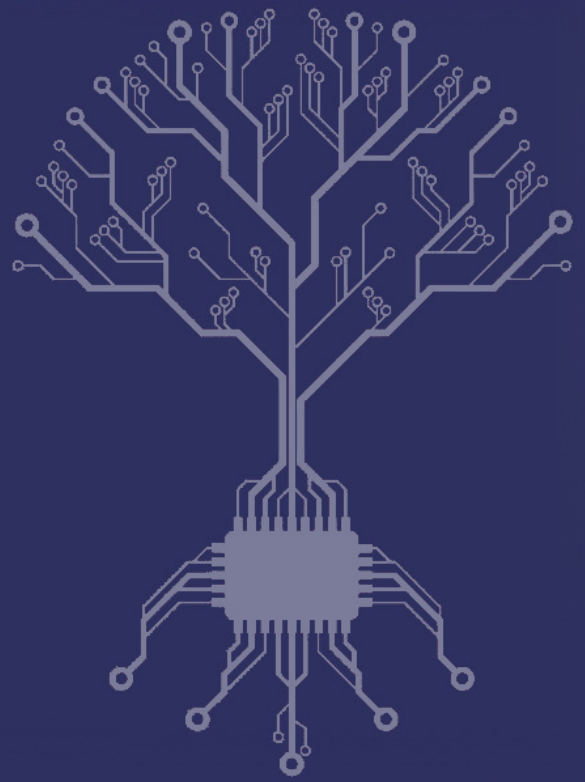
Exemple de questions posées à la base du ministère

- Quel est le nombre d'étudiants inscrits dans chaque centre ?
- Quelles filières sont proposées dans chaque centre ?
- Quels sont les trois meilleurs étudiants de chaque filière du centre Omar Al Khayyam ?
- Quels étudiants ont une moyenne supérieure à la moyenne nationale ?

Objectif

Apprendre à **formuler ces questions** dans un langage compris par le SGBD, afin d'en obtenir automatiquement la réponse.

Modèle relationnel



Principe

Dans le **modèle relationnel**, les données sont structurées sous forme de **tables** à deux dimensions, appelées **relations**.

- Chaque **colonne** (ou attribut) représente une propriété de l'entité (ou de l'association).
- Chaque **ligne** (ou enregistrement) regroupe les valeurs de ces propriétés pour un élément de l'entité (ou de l'association).

Exemple

code_student	first_name	last_name	field	average	city	code_center
1	Ali	Hassan	Mathematics	17.5	Rabat	CP101
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301
4	Laila	Ahmad	Economics	14.4	Rabat	CP102
5	Khalid	Ali	Mathematics	19.2	Salé	CP201

Table – Extrait de la relation *student*

code_center	name	city
CP101	Moulay Youssef	Rabat
CP102	Omar Al Khayyam	Rabat
CP201	Selmane Al Farissi	Salé
CP301	Mohammed VI	Kénitra

Table – Extrait de la relation *center*

Schéma relationnel

Le **schéma** d'une relation est la donnée de son nom et de la liste de ses attributs :

student(*code_student*, *first_name*, *last_name*, *field*, *average*, *city*, *code_center*)

center(*code_center*, *name*, *city*)

Clé primaire

Une **clé primaire** est un attribut (ou un ensemble d'attributs) qui permet d'identifier **de manière unique** chaque ligne de la relation. On la souligne dans le schéma.

- Ici, *code_student* : deux étudiants peuvent avoir le même nom, mais jamais le même identifiant.

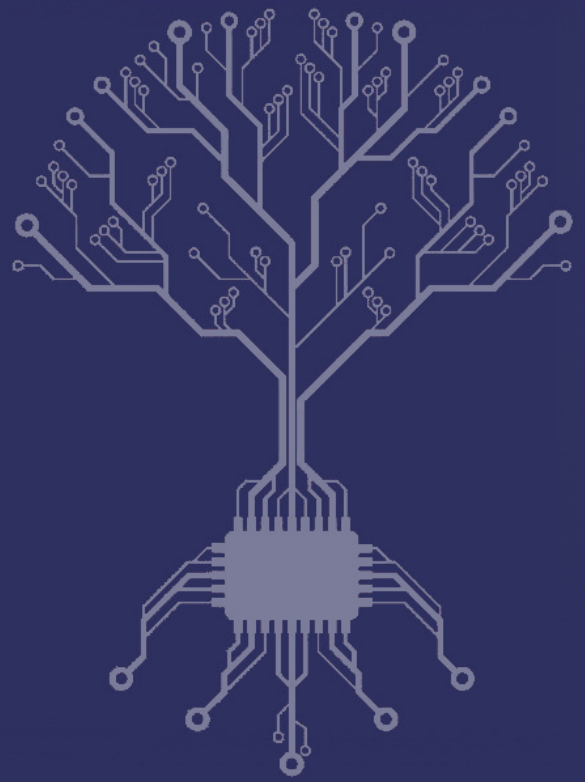
Clé étrangère

Une **clé étrangère** est un attribut d'une relation qui fait référence à la clé primaire d'une autre relation. On la précède de #.

student(*code_student*, *first_name*, *last_name*, *field*, *average*, *city*, #*code_center*)

- *code_center* de la table *student* fait référence à *code_center* de la table *center* : Sarah Mohammad est inscrite au centre CP102, Omar Al Khayyam à Rabat, ce qui représente l'association *s'inscrire à*.

SQL



SQL

Pour communiquer avec un SGBD, on utilise **SQL** (*Structured Query Language*), un langage normalisé qui permet de définir et de manipuler une base de données relationnelle.

On suppose que la base de données est **déjà définie** et on s'intéresse uniquement à son **interrogation**.

Pour interroger une base de données on utilise la commande SELECT. Elle permet d'effectuer, sur les tables, des opérations de l'algèbre relationnelle :

- Récupérer certaines colonnes d'une table (*Projection*) ;
- Récupérer certaines lignes d'une table en fonction de leur contenu (*Sélection*) ;
- Combiner des données venant de plusieurs tables (*Jointure*).

La commande SELECT permet de :

- récupérer certaines colonnes (SELECT) de certaines tables (FROM)
- récupérer certaines lignes (WHERE)
- regrouper certaines lignes (GROUP BY)
- filtrer après ce regroupement (HAVING)
- trier les lignes (ORDER BY)
- limiter le nombre de lignes renvoyées (LIMIT)

Syntaxe

SELECT colonne1, colonne2, ... FROM table ;

ou bien

SELECT DISTINCT colonne1, colonne2, ... FROM table ;

Le mot clé DISTINCT permet de supprimer les doublons.

Exemple

On reprend la relation *student* :

student(*code_student*, *first_name*, *last_name*, *field*, *average*, *city*, #*code_center*)

Exemples

- Pour afficher les prénoms et les noms des étudiants :
  - SELECT first_name, last_name FROM student ;
- Pour afficher les villes (sans doublons) des étudiants :
  - SELECT DISTINCT city FROM student ;
- Pour afficher toutes les informations des étudiants :
  - SELECT * FROM student ;

Syntaxe

SELECT colonne1, colonne2, ... FROM table WHERE conditions

Exemples

- Afficher les prénoms et les noms des étudiants qui habitent à Rabat :
  - SELECT first_name, last_name FROM student WHERE city = *'Rabat'* ;
- Afficher les prénoms, les noms et les moyennes des étudiants qui habitent à *'Salé'* ou à *'Kénitra'* :
  - SELECT first_name, last_name, average FROM student WHERE city = *'Salé'* OR city = *'Kénitra'* ;
  - SELECT first_name, last_name, average FROM student WHERE city IN (*'Salé'*, *'Kénitra'*) ;

Exemple

- Afficher les prénoms et les noms des étudiants dont la moyenne est supérieure ou égale à 12 :
  - SELECT first_name, last_name FROM student WHERE average >= 12 ;

Il est possible de trier les données sur une ou plusieurs colonnes par ordre ascendant (par défaut) ou descendant.

Syntaxe

... ORDER BY colonne1 [ASC ou DESC], colonne2 [ASC ou DESC], ... ;

Exemple

- Afficher les prénoms, les noms et les moyennes des étudiants triées par ordre ascendant :
  - SELECT first_name, last_name, average FROM student ORDER BY average ;

Pour limiter le nombre de lignes renvoyées par une requête, on utilise la commande :

Syntaxe

... LIMIT nombre1 OFFSET nombre2 ;

qui permet de limiter le nombre de lignes à *nombre1*, et de sauter *nombre2* lignes avant de renvoyer le résultat.

Exemple

- Afficher les prénoms et les noms des trois meilleurs étudiants :
  - SELECT first_name, last_name FROM student ORDER BY average DESC LIMIT 3 ;

- SQL dispose aussi d'un ensemble de fonctions d'agrégation comme :
  - SUM : pour calculer la somme des valeurs.
  - AVG : pour calculer la moyenne des valeurs.
  - COUNT : pour compter le nombre de lignes concernées.
  - MAX : pour récupérer la plus grande valeur.
  - MIN : pour récupérer la plus petite valeur.

Syntaxe

```
SELECT fonction(nomColonne) [AS alias], ... FROM ... ;
```

Exemples

- Afficher la note maximale, la note minimale et la moyenne des notes des étudiants :
  - SELECT MAX(average), MIN(average), AVG(average) FROM student ;
- Afficher le nombre des étudiants :
  - SELECT COUNT(*) AS *'Number of students'* FROM student ;
    Renvoie le nombre de lignes de la table.
- Afficher le nombre de villes des étudiants :
  - SELECT COUNT(DISTINCT city) FROM student ;
    Pour s'assurer qu'on compte une ville une seule fois.

En utilisant une agrégation, on peut regrouper les lignes d'une table en fonction des valeurs d'une ou plusieurs colonnes à l'aide du mot-clé GROUP BY.

Exemples

- Afficher la note maximale de chaque filière :
  - SELECT field, MAX(average) FROM student GROUP BY field ;
- Afficher la note maximale de chaque filière de chaque ville :
  - SELECT field, city, MAX(average) FROM student GROUP BY field, city ;

Points importants :

- Toutes les colonnes non agrégées dans le SELECT doivent être incluses dans le GROUP BY.
- Les colonnes listées après GROUP BY définissent les groupes. Chaque groupe contiendra les lignes ayant les mêmes valeurs dans ces colonnes.

Il est possible d'effectuer une sélection après une agrégation, en ajoutant une condition à laquelle chaque groupe doit répondre.

Syntaxe

```
SELECT ... FROM table GROUP BY ... HAVING condition ;
```

Exemple

- On ajoute à l'exemple précédent la condition de n'afficher que les notes maximales ≥ 10 :
  - SELECT field, MAX(average) FROM student GROUP BY field HAVING MAX(average) >= 10 ;